# Laboratorium 5: Hermetyzacja w języku Java

Dominik Delimata

Informatyka

1 rok

Laboratorium nr 1

**ZADANIA**

**Zadanie 1.** Dla wszystkich zadań z poprzedniego konspektu (Lab. 4.) dodać hermetyzację

**DaneOsobowe**

DaneOsobowe.java

```
package daneosobowe;

public class DaneOsobowe {
    public static void main(String[] args) {
        Osoba osoba1 = new Osoba("Jan", "Kowalski", 15);
        osoba1.PokazDane();

        System.out.println("-----------------------------------------------
---");
        Osoba osoba2 = new Osoba("Adam", "Nowak");
        osoba2.PokazDane();

        System.out.println("-----------------------------------------------
---");
        osoba2.setImie("Stefan");
        osoba2.setWiek(70);
        osoba2.PokazDane();

        System.out.println("-----------------------------------------------
---");
        Osoba osoba3 = new Osoba();
        osoba3.setImie("Anna");
        osoba3.setNazwisko("Wiśniewska");
        osoba3.setWiek(45);
        osoba3.PokazDane();
    }
}
```

Osoba.java

```
package daneosobowe;
public class Osoba {
    private String imie;
    private String nazwisko;
    private int wiek;

    public Osoba(String imie, String nazwisko, int wiek) {
```

```
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.wiek = wiek;
    }

    public Osoba(String imie, String nazwisko) {
        this.imie = imie;
        this.nazwisko = nazwisko;
    }

    public Osoba() {
    }

    public String getImie() {
        return imie;
    }

    public String getNazwisko() {
        return nazwisko;
    }

    public int getWiek() {
        return wiek;
    }

    public void setImie(String imie) {
        this.imie = imie;
    }

    public void setNazwisko(String nazwisko) {
        this.nazwisko = nazwisko;
    }

    public void setWiek(int wiek) {
        if (wiek >= 0) {
            this.wiek = wiek;
        } else {
            System.out.println("Błąd: Wiek nie może być ujemny!");
        }
    }

    public void PokazDane() {
        System.out.println("Osoba");
        System.out.println("imię: " + this.imie);
        System.out.println("nazwisko: " + this.nazwisko);
        System.out.println("wiek: " + this.wiek + " lat");
    }
}
```

Dzięki hermetyzacji nasz kod zyska na bezpieczeństwie danych: nikt przypadkowo nie zmieni wieku na -50. Jeśli spróbuje wykonać osoba3.setWiek(-50), nasza klasa zablokuje tę zmianę i wypisze komunikat błędu. Oraz na kontroli: Klasa Osoba ma teraz pełną władzę nad tym, co dzieje się z jej danymi.

**STUDENT**

Student.java

```
package student;

public class Student {
    // 1. Pola zmienione na private
    private String imie;
    private String nazwisko;
    private int nr_indeksu;
    private String nazwa_specjalnosci;
    private int rok_studiow;

    public Student(String imie, String nazwisko, int nr_indeksu, String
nazwa_specjalnosci, int rok_studiow) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.nr_indeksu = nr_indeksu;
        this.nazwa_specjalnosci = nazwa_specjalnosci;
        this.rok_studiow = rok_studiow;
    }

    public Student(String imie, String nazwisko, int nr_indeksu) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.nr_indeksu = nr_indeksu;
    }

    public Student(String imie, String nazwisko, String nazwa_specjalnosci)
{
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.nazwa_specjalnosci = nazwa_specjalnosci;
    }

    public Student(String imie, String nazwisko, int nr_indeksu, int
rok_studiow) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.nr_indeksu = nr_indeksu;
        this.rok_studiow = rok_studiow;
    }

    public String getImie() {
        return imie;
    }

    public String getNazwisko() {
        return nazwisko;
    }

    public int getNr_indeksu() {
        return nr_indeksu;
    }

    public String getNazwa_specjalnosci() {
        return nazwa_specjalnosci;
    }

    public int getRok_studiow() {
```

```
        return rok_studiow;
    }

    public void setImie(String imie) {
        this.imie = imie;
    }

    public void setNazwisko(String nazwisko) {
        this.nazwisko = nazwisko;
    }

    public void setNr_indeksu(int nr_indeksu) {
        if (nr_indeksu > 0) {
            this.nr_indeksu = nr_indeksu;
        } else {
            System.out.println("Błąd: Numer indeksu musi być dodatni!");
        }
    }

    public void setNazwa_specjalnosci(String nazwa_specjalnosci) {
        this.nazwa_specjalnosci = nazwa_specjalnosci;
    }

    public void setRok_studiow(int rok_studiow) {
        if (rok_studiow > 0 && rok_studiow <= 5) {
            this.rok_studiow = rok_studiow;
        } else {
            System.out.println("Błąd: Rok studiów powinien mieścić się w
przedziale od 1 do 5.");
        }
    }

    public void pokazDane() {
        System.out.println("=== Dane studenta ===");
        System.out.println("Imię:                " + this.imie);
        System.out.println("Nazwisko:            " + this.nazwisko);
        System.out.println("Nr indeksu:          " + this.nr_indeksu);
        System.out.println("Specjalność:         " +
this.nazwa_specjalnosci);
        System.out.println("Rok studiów:         " + this.rok_studiow);
    }
}
```

StudentMain.java

```
package student;

import java.util.Scanner;

public class StudenciMain {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Wprowadzanie danych studenta ===\n");

        System.out.print("Podaj imię: ");
        String imie = sc.nextLine();

        System.out.print("Podaj nazwisko: ");
```

```java
        String nazwisko = sc.nextLine();

        System.out.print("Podaj nr indeksu: ");
        int nr_indeksu = sc.nextInt();
        sc.nextLine();

        System.out.print("Podaj nazwę specjalności: ");
        String specjalnosc = sc.nextLine();

        System.out.print("Podaj rok studiów: ");
        int rok = sc.nextInt();

        // Tworzenie obiektu za pomocą konstruktora
        student.Student student = new student.Student(imie, nazwisko,
nr_indeksu, specjalnosc, rok);

        System.out.println("\n--- Wprowadzone dane ---");
        student.pokazDane();

        // Przykład użycia settera - symulacja awansu na kolejny rok
        System.out.println("\n--- Symulacja: Student zdaje na kolejny rok -
--");
        int aktualnyRok = student.getRok_studiow(); // Pobieramy aktualny
rok getterem
        student.setRok_studiow(aktualnyRok + 1);    // Zwiększamy o 1
setterem
        student.pokazDane();

        sc.close();
    }
}
```

```
Run   StudenciMain

C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-javaage
=== Wprowadzanie danych studenta ===

Podaj imię: Dominik
Podaj nazwisko: Delimata
Podaj nr indeksu: 141319
Podaj nazwę specjalności: Informatyka
Podaj rok studiów: 1

--- Wprowadzone dane ---
=== Dane studenta ===
Imię:               Dominik
Nazwisko:           Delimata
Nr indeksu:         141319
Specjalność:        Informatyka
Rok studiów:        1

--- Symulacja: Student zdaje na kolejny rok ---
=== Dane studenta ===
Imię:               Dominik
Nazwisko:           Delimata
Nr indeksu:         141319
Specjalność:        Informatyka
Rok studiów:        2

Process finished with exit code 0
```

**ObliczanieFigur**

Kolo.java

```
package obliczaniefigur;

public class Kolo {
    private double promien;

    public Kolo(double promien) {
        setPromien(promien); // Używamy settera w konstruktorze dla
spójności walidacji
    }

    public double getPromien() {
        return promien;
    }

    public void setPromien(double promien) {
        if (promien > 0) this.promien = promien;
        else System.out.println("Błąd: Promień musi być większy od 0!");
    }

    public double pole() {
        return Math.PI * promien * promien;
    }

    public double obwod() {
        return 2 * Math.PI * promien;
    }

    public void pokazDane() {
        System.out.println("=== Koło ===");
        System.out.printf("Promień:  %.2f%n", promien);
        System.out.printf("Pole:     %.2f%n", pole());
        System.out.printf("Obwód:    %.2f%n", obwod());
    }
}
```

Kwadrat.java

```
package obliczaniefigur;

public class Kwadrat {
    private double bok;

    public Kwadrat(double bok) {
        setBok(bok);
    }

    public double getBok() {
        return bok;
    }

    public void setBok(double bok) {
        if (bok > 0) this.bok = bok;
        else System.out.println("Błąd: Bok musi być większy od 0!");
    }
```

```java
    public double pole() {
        return bok * bok;
    }

    public double obwod() {
        return 4 * bok;
    }

    public void pokazDane() {
        System.out.println("=== Kwadrat ===");
        System.out.printf("Bok:      %.2f%n", bok);
        System.out.printf("Pole:     %.2f%n", pole());
        System.out.printf("Obwód:    %.2f%n", obwod());
    }
}
```

Prostokat.java

```java
package obliczaniefigur;

public class Prostokat {
    private double dlugosc;
    private double szerokosc;

    public Prostokat(double dlugosc, double szerokosc) {
        setDlugosc(dlugosc);
        setSzerokosc(szerokosc);
    }

    public double getDlugosc() { return dlugosc; }
    public double getSzerokosc() { return szerokosc; }

    public void setDlugosc(double dlugosc) {
        if (dlugosc > 0) this.dlugosc = dlugosc;
        else System.out.println("Błąd: Długość musi być większa od 0!");
    }

    public void setSzerokosc(double szerokosc) {
        if (szerokosc > 0) this.szerokosc = szerokosc;
        else System.out.println("Błąd: Szerokość musi być większa od 0!");
    }

    public double pole() {
        return dlugosc * szerokosc;
    }

    public double obwod() {
        return 2 * (dlugosc + szerokosc);
    }

    public void pokazDane() {
        System.out.println("=== Prostokąt ===");
        System.out.printf("Długość:    %.2f%n", dlugosc);
        System.out.printf("Szerokość:  %.2f%n", szerokosc);
        System.out.printf("Pole:       %.2f%n", pole());
        System.out.printf("Obwód:      %.2f%n", obwod());
    }
}
```

Szescian.java

```
package obliczaniefigur;

public class Szescian {
    private double bok;

    public Szescian(double bok) {
        setBok(bok);
    }

    public double getBok() {
        return bok;
    }

    public void setBok(double bok) {
        if (bok > 0) this.bok = bok;
        else System.out.println("Błąd: Bok musi być większy od 0!");
    }

    public double polePowierzchni() {
        return 6 * bok * bok;
    }

    public double objetosc() {
        return bok * bok * bok;
    }

    public void pokazDane() {
        System.out.println("=== Sześcian ===");
        System.out.printf("Bok:               %.2f%n", bok);
        System.out.printf("Pole powierzchni: %.2f%n", polePowierzchni());
        System.out.printf("Objętość:          %.2f%n", objetosc());
    }
}
```

Prostopadloscian.java

```
package obliczaniefigur;

public class Prostopadloscian {
    private double dlugosc;
    private double szerokosc;
    private double wysokosc;

    public Prostopadloscian(double dlugosc, double szerokosc, double
wysokosc) {
        setDlugosc(dlugosc);
        setSzerokosc(szerokosc);
        setWysokosc(wysokosc);
    }

    public double getDlugosc() { return dlugosc; }
    public double getSzerokosc() { return szerokosc; }
    public double getWysokosc() { return wysokosc; }

    public void setDlugosc(double dlugosc) {
        if (dlugosc > 0) this.dlugosc = dlugosc;
        else System.out.println("Błąd: Długość musi być większa od 0!");
    }
```

```java
    public void setSzerokosc(double szerokosc) {
        if (szerokosc > 0) this.szerokosc = szerokosc;
        else System.out.println("Błąd: Szerokość musi być większa od 0!");
    }

    public void setWysokosc(double wysokosc) {
        if (wysokosc > 0) this.wysokosc = wysokosc;
        else System.out.println("Błąd: Wysokość musi być większa od 0!");
    }

    public double polePowierzchni() {
        return 2 * (dlugosc * szerokosc + szerokosc * wysokosc + dlugosc *
wysokosc);
    }

    public double objetosc() {
        return dlugosc * szerokosc * wysokosc;
    }

    public void pokazDane() {
        System.out.println("=== Prostopadłościan ===");
        System.out.printf("Długość:          %.2f%n", dlugosc);
        System.out.printf("Szerokość:        %.2f%n", szerokosc);
        System.out.printf("Wysokość:         %.2f%n", wysokosc);
        System.out.printf("Pole powierzchni: %.2f%n", polePowierzchni());
        System.out.printf("Objętość:         %.2f%n", objetosc());
    }
}
```

Kula.java

```java
package obliczaniefigur;

public class Kula {
    private double promien;

    public Kula(double promien) {
        setPromien(promien);
    }

    public double getPromien() {
        return promien;
    }

    public void setPromien(double promien) {
        if (promien > 0) this.promien = promien;
        else System.out.println("Błąd: Promień musi być większy od 0!");
    }

    public double polePowierzchni() {
        return 4 * Math.PI * promien * promien;
    }

    public double objetosc() {
        return (4.0 / 3.0) * Math.PI * promien * promien * promien;
    }

    public void pokazDane() {
        System.out.println("=== Kula ===");
```

```
        System.out.printf("Promień:           %.2f%n", promien);
        System.out.printf("Pole powierzchni: %.2f%n", polePowierzchni());
        System.out.printf("Objętość:          %.2f%n", objetosc());
    }
}
```

Stozek.java

```
package obliczaniefigur;

public class Stozek {
    private double promien;
    private double wysokosc;

    public Stozek(double promien, double wysokosc) {
        setPromien(promien);
        setWysokosc(wysokosc);
    }

    public double getPromien() { return promien; }
    public double getWysokosc() { return wysokosc; }

    public void setPromien(double promien) {
        if (promien > 0) this.promien = promien;
        else System.out.println("Błąd: Promień musi być większy od 0!");
    }

    public void setWysokosc(double wysokosc) {
        if (wysokosc > 0) this.wysokosc = wysokosc;
        else System.out.println("Błąd: Wysokość musi być większa od 0!");
    }

    public double tworzaca() {
        return Math.sqrt(promien * promien + wysokosc * wysokosc);
    }

    public double polePowierzchni() {
        return Math.PI * promien * (promien + tworzaca());
    }

    public double objetosc() {
        return (1.0 / 3.0) * Math.PI * promien * promien * wysokosc;
    }

    public void pokazDane() {
        System.out.println("=== Stożek ===");
        System.out.printf("Promień:           %.2f%n", promien);
        System.out.printf("Wysokość:          %.2f%n", wysokosc);
        System.out.printf("Tworząca:          %.2f%n", tworzaca());
        System.out.printf("Pole powierzchni: %.2f%n", polePowierzchni());
        System.out.printf("Objętość:          %.2f%n", objetosc());
    }
}
```

Hermetyzacja znacząco podniosła bezpieczeństwo i stabilność Twojego kodu kalkulatora figur. Wcześniej każdy mógł stworzyć Kwadrat o boku -5, co dałoby poprawne matematycznie pole (25), ale nielogiczny ujemny obwód (-20). Teraz, klasy same kontrolują swój stan wewnątrz, a program stał się "idiotoodporny" na wprowadzanie niewłaściwych wymiarów.

**WprowadzDane**

Student.java

```
package wprowadzdane;

public class Student {
    // Pola są teraz prywatne
    private String imie;
    private String nazwisko;
    private int nr_indeksu;
    private String nazwa_specjalnosci;
    private int rok_studiow;

    public Student(String imie, String nazwisko, int nr_indeksu, String
nazwa_specjalnosci, int rok_studiow) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.nr_indeksu = nr_indeksu;
        this.nazwa_specjalnosci = nazwa_specjalnosci;
        this.rok_studiow = rok_studiow;
    }

    // --- GETTERY ---
    public String getImie() { return imie; }
    public String getNazwisko() { return nazwisko; }
    public int getNr_indeksu() { return nr_indeksu; }
    public String getNazwa_specjalnosci() { return nazwa_specjalnosci; }
    public int getRok_studiow() { return rok_studiow; }

    // --- SETTERY ---
    public void setImie(String imie) { this.imie = imie; }
    public void setNazwisko(String nazwisko) { this.nazwisko = nazwisko; }
    public void setNr_indeksu(int nr_indeksu) { this.nr_indeksu =
nr_indeksu; }
    public void setNazwa_specjalnosci(String nazwa_specjalnosci) {
this.nazwa_specjalnosci = nazwa_specjalnosci; }
    public void setRok_studiow(int rok_studiow) { this.rok_studiow =
rok_studiow; }

    public void pokazDane() {
        System.out.println("Imię: " + imie);
        System.out.println("Nazwisko: " + nazwisko);
        System.out.println("Nr indeksu: " + nr_indeksu);
        System.out.println("Nazwa specjalności: " + nazwa_specjalnosci);
        System.out.println("Rok studiów: " + rok_studiow);
    }
}
```

BazaStudentow.java

```
package wprowadzdane;

import java.util.Scanner;

public class BazaStudentow {

    // Zmienne bazy są teraz prywatne
    private static final int ROZMIAR = 100;
    private Student[] tablica = new Student[ROZMIAR];
    private Scanner sc = new Scanner(System.in);

    public void inicjalizuj() {
        for (int i = 0; i < ROZMIAR; i++) {
            tablica[i] = new Student("", "", 0, "", 0);
        }
        System.out.println("Tablica zainicjalizowana wartościami
domyślnymi.");
    }

    private Student wczytajStudenta() {
        System.out.print("Imię: ");
        String imie = sc.nextLine();

        System.out.print("Nazwisko: ");
        String nazwisko = sc.nextLine();

        System.out.print("Nr indeksu: ");
        int nr = sc.nextInt();
        sc.nextLine();

        System.out.print("Specjalność: ");
        String spec = sc.nextLine();

        System.out.print("Rok studiów: ");
        int rok = sc.nextInt();
        sc.nextLine();

        return new Student(imie, nazwisko, nr, spec, rok);
    }

    public void wprowadzDane() {
        System.out.print("Podaj indeks (0-" + (ROZMIAR - 1) + "): ");
        int idx = sc.nextInt();
        sc.nextLine();

        if (idx < 0 || idx >= ROZMIAR) {
            System.out.println("Niepoprawny indeks!");
            return;
        }

        // .imie -> getImie
        if (!tablica[idx].getImie().equals("")) {
            System.out.println("Pod tym indeksem już są dane! Użyj opcji
Edycja.");
            return;
        }

        tablica[idx] = wczytajStudenta();
        System.out.println("Dane zapisane pod indeksem " + idx);
```

```
    }

    public void edytujDane() {
        System.out.print("Podaj indeks do edycji (0-" + (ROZMIAR - 1) + "): 
");
        int idx = sc.nextInt();
        sc.nextLine();

        if (idx < 0 || idx >= ROZMIAR) {
            System.out.println("Niepoprawny indeks!");
            return;
        }

        System.out.println("Aktualne dane:");
        tablica[idx].pokazDane();
        System.out.println("Wprowadź nowe dane:");
        tablica[idx] = wczytajStudenta();
        System.out.println("Dane zaktualizowane pod indeksem " + idx);
    }

    public void usunDane() {
        System.out.print("Podaj indeks do usunięcia (0-" + (ROZMIAR - 1) + 
"): ");
        int idx = sc.nextInt();
        sc.nextLine();

        if (idx < 0 || idx >= ROZMIAR) {
            System.out.println("Niepoprawny indeks!");
            return;
        }

        tablica[idx] = new Student("", "", 0, "", 0);
        System.out.println("Dane pod indeksem " + idx + " zostały 
usunięte.");
    }

    public void wyswietlJeden() {
        System.out.print("Podaj indeks (0-" + (ROZMIAR - 1) + "): ");
        int idx = sc.nextInt();
        sc.nextLine();

        if (idx < 0 || idx >= ROZMIAR) {
            System.out.println("Niepoprawny indeks!");
            return;
        }

        System.out.println("=== Student [" + idx + "] ===");
        tablica[idx].pokazDane();
    }

    public void wyswietlWszystkich() {
        System.out.println("=== Wszyscy studenci ===");
        for (int i = 0; i < ROZMIAR; i++) {
            // ZMIANA: Zamiast .imie używamy .getImie()
            if (!tablica[i].getImie().equals("")) {
                System.out.println("--- Indeks [" + i + "] ---");
                tablica[i].pokazDane();
                System.out.println();
            }
        }
    }
```

```java
    public void wyswietlZakres() {
        System.out.print("Podaj indeks początkowy: ");
        int od = sc.nextInt();
        System.out.print("Podaj indeks końcowy: ");
        int do_ = sc.nextInt();
        sc.nextLine();

        if (od < 0 || do_ >= ROZMIAR || od > do_) {
            System.out.println("Niepoprawny zakres!");
            return;
        }

        System.out.println("=== Studenci od [" + od + "] do [" + do_ + "]
===");
        for (int i = od; i <= do_; i++) {
            System.out.println("--- Indeks [" + i + "] ---");
            tablica[i].pokazDane();
            System.out.println();
        }
    }
}
```

Plik WprowadzDane.java pozostał bez zmian, ponieważ od samego początku poprawnie korzystał tylko z publicznych metod klasy BazaStudentow, nie odwołując się do jej zmiennych wewnętrznych

Zabezpieczenie Klasy Student: Zmieniłem modyfikatory pól klasy Student na private, a dostęp do nich otworzyłem wyłącznie przez metody typu getter i setter. Dzięki temu pola nie mogą być przypadkowo zmienione z zewnątrz.

Adaptacja Kodu Bazy: W klasie BazaStudentow, w miejscach, gdzie wcześniej kod miał bezpośredni wgląd do imienia studenta (np. sprawdzając if (!tablica[idx].imie.equals("")), teraz bezpiecznie korzysta z publicznego interfejsu (czyli z metody getImie()).

Hermetyzacja samej Bazy: Dodatkowo ukryłem same "bebechy" systemu bazodanowego (tablica, ROZMIAR, instancja klasy Scanner) przypisując im modyfikator private. Zapobiega to sytuacji, w której jakakolwiek inna klasa (np. WprowadzDane) zepsułaby działanie programu podmieniając całą wewnętrzną tablicę w czasie działania aplikacji.

**Zadanie 2**. Zaprojektuj i utwórz klasę, która opisuje pozycję książkową w księgarni. Książki cechują się następującymi właściwościami: tytuł, autor, liczba stron, rok wydania, cena. Zastosuj enkapsulację pól w klasie, przy czym uwzględnij, że tylko cena książki może się zmieniać w czasie, a pozostałe atrybuty są niezmienne (tylko do odczytu). Utwórz przykładowe obiekty i pokaż w jaki sposób można dostać się do pól obiektu poprzez metody.

```
public class Ksiazka {

    // 1. POLA KLASY (Zmienne)
    private String tytul;
    private String autor;
    private int liczbaStron;
    private int rokWydania;
    private double cena;

    // 2. KONSTRUKTOR
    public Ksiazka(String tytul, String autor, int liczbaStron, int
rokWydania, double cena) {
        this.tytul = tytul;
        this.autor = autor;
        this.liczbaStron = liczbaStron;
        this.rokWydania = rokWydania;
        this.cena = cena;
    }

    // 3. GETTERY (Metody do odczytu)
    public String getTytul() {
        return tytul; // po prostu zwraca tytuł
    }

    public String getAutor() {
        return autor;
    }

    public int getLiczbaStron() {
        return liczbaStron;
    }

    public int getRokWydania() {
        return rokWydania;
    }

    public double getCena() {
        return cena;
    }

    // 4. SETTERY (Metody do zmiany wartości)
    public void setCena(double nowaCena) {
        if (nowaCena > 0) {
            this.cena = nowaCena;
        } else {
            System.out.println("Błąd! Cena nie może być ujemna ani
zerowa.");
        }
    }

    // 5. METODA MAIN - tutaj testuję czy mój kod działa
```

```java
    public static void main(String[] args) {

        System.out.println("=== Witaj w mojej Księgarni! ===\n");

        // Przyklady ksiazek
        Ksiazka ksiazka1 = new Ksiazka("Władca Pierścieni", "J.R.R.
Tolkien", 1200, 1954, 89.99);
        Ksiazka ksiazka2 = new Ksiazka("Harry Potter", "J.K. Rowling", 320,
1997, 45.50);


        System.out.println("Dane pierwszej książki:");
        System.out.println("Tytuł: " + ksiazka1.getTytul());
        System.out.println("Autor: " + ksiazka1.getAutor());
        System.out.println("Rok wydania: " + ksiazka1.getRokWydania());
        System.out.println("Cena początkowa: " + ksiazka1.getCena() + "
zł\n");

        System.out.println("Zmieniamy cenę książki nr 2...");

        ksiazka2.setCena(55.00); // Używamy naszego publicznego setera

        System.out.println("Nowa cena za '" + ksiazka2.getTytul() + "' to:
" + ksiazka2.getCena() + " zł");
    }
}
```

Run  Ksiazka

```
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe "-ja
=== Witaj w mojej Księgarni! ===

Dane pierwszej książki:
Tytuł: Władca Pierścieni
Autor: J.R.R. Tolkien
Rok wydania: 1954
Cena początkowa: 89.99 zł

Zmieniamy cenę książki nr 2...
Nowa cena za 'Harry Potter' to: 55.0 zł

Process finished with exit code 0
```

**Zadanie 3.** Zaprojektuj klasę symulującą działanie struktury stosu. W ramach klasy występują następujące prywatne pola: n-elementowa tablica liczb całkowitych reprezentująca stos, wskaźnik stosu jako liczba całkowita. Wskaźnik stosu wskazuje element ze szczytu stosu. Utwórz metody: umieszczającą element na szczycie stosu „push" oraz ściągającą wartość ze szczytu stosu „pop". Zadbaj o to aby w razie przepełnienia stosu nie można było umieścić wartości na stosie, a w razie stosu pustego nie można było ściągnąć danej ze stosu. Zauważ, że instrukcje „push" i „pop" zachowują się jak „get" i „set". Utwórz konstruktor, którego argumentem jest rozmiar stosu. W ciele konstruktora dokonaj inicjalizacji n-elementowej tablicy reprezentującej stos. Pamiętaj, że instrukcja push i pop zmieniają odpowiednio wskaźnik stosu.

```
public class Stos {
    // 1. POLA PRYWATNE (Enkapsulacja)
    private int[] tablica;
    private int wskaznik;

    // 2. KONSTRUKTOR
    public Stos(int rozmiar) {
        tablica = new int[rozmiar];
        wskaznik = -1;
    }

    // 3. METODA PUSH (wstawianie na szczyt)
    public void push(int element) {
        if (wskaznik == tablica.length - 1) {
            System.out.println("BŁĄD: Stos pełny! Nie mogę wcisnąć
elementu: " + element);
        } else {
            wskaznik++;
            tablica[wskaznik] = element;
            System.out.println("Push: umieszczono na stosie element " +
element);
        }
    }

    // 4. METODA POP (ściąganie ze szczytu)
    public int pop() {
        if (wskaznik == -1) {
            System.out.println("BŁĄD: Stos jest pusty! Nie ma czego
zdjąć.");
            return -1;
        } else {
            int pobranyElement = tablica[wskaznik];
            wskaznik--;
            System.out.println("Pop: ściągnięto ze stosu element " +
pobranyElement);
            return pobranyElement;
        }
    }

    // 5. METODA MAIN - do przetestowania mojego kodu
    public static void main(String[] args) {
        System.out.println("=== Mój testowy stos ===");

        Stos mojStos = new Stos(3);

        mojStos.push(10); // Element 1
```

```java
        mojStos.push(20); // Element 2
        mojStos.push(30); // Element 3

        mojStos.push(40);

        System.out.println("--------------------");
        mojStos.pop();
        mojStos.pop();
        mojStos.pop();
        mojStos.pop();
    }
}
```

```
C:\Users\Admin\.jdks\openjdk-25.0.2\bin\java.exe
=== Mój testowy stos ===
Push: umieszczono na stosie element 10
Push: umieszczono na stosie element 20
Push: umieszczono na stosie element 30
BŁĄD: Stos pełny! Nie mogę wcisnąć elementu: 40
--------------------
Pop: ściągnięto ze stosu element 30
Pop: ściągnięto ze stosu element 20
Pop: ściągnięto ze stosu element 10
BŁĄD: Stos jest pusty! Nie ma czego zdjąć.

Process finished with exit code 0
```

Użyliśmy zwykłej tablicy, żeby symulować strukturę zwaną stosem (LIFO). Wszystko opiera się na jednej dodatkowej zmiennej wskaznik (Top). Tablica tak naprawdę cały czas jest w pamięci, ale my dzięki wskaźnikowi udajemy, że wiemy, gdzie ten nasz "stos" się kończy.

Metoda push: Trzeba pamiętać, by najpierw podnieść wskaźnik (wskaznik++), a dopiero potem wpisać wartość pod ten nowy indeks tablicy.

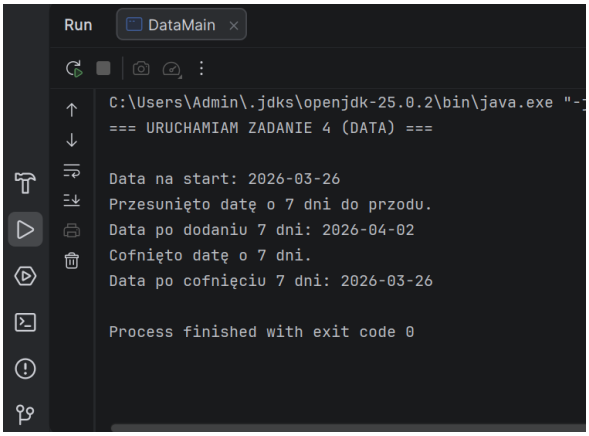
Z kolei w metodzie pop kolejność jest odwrotna! Najpierw kopiujemy wartość ze szczytu do pomocniczej zmiennej, potem obniżamy wskaźnik (wskaznik--), a na samym końcu zwracamy skopiowaną wcześniej wartość (return).

**Zadanie 4.** Zdefiniuj klasę opisującą datę. Zastanów się nad wyborem wewnętrznej reprezentacji dat. Zdefiniuj metody pozwalające na odczytywanie bieżącej daty i przestawianie jej o jeden tydzień w przód i w tył. Zadbaj o dobranie odpowiednich modyfikatorów dostępu do składowych.

```
import java.time.LocalDate;

public class MojaData {

    // 1. REPREZENTACJA WEWNĘTRZNA
    // Używam gotowej klasy LocalDate. Ukrywam ją (private), żeby
    // nikt nie zmienił jej bez użycia moich metod.
    private LocalDate aktualnaData;

    // 2. KONSTRUKTOR
    // Kiedy tworzymy nowy obiekt, ustawiamy konkretną datę.
    public MojaData(int rok, int miesiac, int dzien) {
        this.aktualnaData = LocalDate.of(rok, miesiac, dzien);
    }

    // 3. METODA DO ODCZYTYWANIA DATY (Getter)
    // Pozwala tylko na podgląd, bez możliwości modyfikacji z zewnątrz.
    public LocalDate getBiezacaData() {
        return aktualnaData;
    }

    // 4. METODA: PRZESTAWIANIE O TYDZIEŃ W PRZÓD
    public void przesunTydzienWPrzod() {
        // LocalDate ma super wygodną metodę plusWeeks().
        // Sama ogarnia, czy przeskakujemy na nowy miesiąc!
        this.aktualnaData = this.aktualnaData.plusWeeks(1);
        System.out.println("Przesunięto datę o 7 dni do przodu.");
    }

    // 5. METODA: PRZESTAWIANIE O TYDZIEŃ W TYŁ
    public void przesunTydzienWTyl() {
        // Analogicznie, używamy minusWeeks()
        this.aktualnaData = this.aktualnaData.minusWeeks(1);
        System.out.println("Cofnięto datę o 7 dni.");
    }
}
```

```
public class DataMain {

    // Tutaj zaczyna się działanie pierwszego programu
    public static void main(String[] args) {

        System.out.println("=== URUCHAMIAM ZADANIE 4 (DATA) ===\n");

        // 1. Tworzymy nasz obiekt daty - wpisujemy np. dzisiejszą datę
        MojaData testowaData = new MojaData(2026, 3, 26);

        // 2. Odczytujemy i wyświetlamy datę początkową (używamy gettera!)
        System.out.println("Data na start: " +
testowaData.getBiezacaData());

        // 3. Testujemy przesuwanie o tydzień do przodu
        testowaData.przesunTydzienWPrzod();
```

```
        System.out.println("Data po dodaniu 7 dni: " +
testowaData.getBiezacaData());

        // 4. Testujemy przesuwanie o tydzień do tyłu
        testowaData.przesunTydzienWTyl();
        System.out.println("Data po cofnięciu 7 dni: " +
testowaData.getBiezacaData());
    }
}
```




Nie warto wymyślać koła na nowo. Gdybym stworzył datę opartą o int dzien, int miesiac, napisanie kodu, który poprawnie dodaje tydzień w okolicach 28 lutego (zwłaszcza w roku przestępnym), byłoby bardzo skomplikowane. Użycie wbudowanej klasy java.time.LocalDate ułatwia sprawę.

**Zadanie 5.** Zdefiniuj klasę Pracownik (podobnie do klasy osoba z wykładu, dodając jeszcze tekstową informację o zajmowanym przez pracownika stanowisku). Następnie zdefiniuj klasę Firma, która by przechowywała w tablicy spis wszystkich pracowników (możesz założyć, że liczba pracowników nie przekracza 100). Zdefiniuj metody dodawania nowych pracowników do firmy oraz wypisywania aktualnego spisu pracowników. Zastanów się, jak przy tym ostatnim zadaniu rozdzielić odpowiedzialności pomiędzy obie klasy.

```
public class Pracownik {
    // Pola prywatne - enkapsulacja!
    private String imie;
    private String nazwisko;
    private String stanowisko;

    // Konstruktor
    public Pracownik(String imie, String nazwisko, String stanowisko) {
        this.imie = imie;
        this.nazwisko = nazwisko;
        this.stanowisko = stanowisko;
    }

    public void wypiszDanePracownika() {
        System.out.println("Pracownik: " + imie + " " + nazwisko + " |
Stanowisko: " + stanowisko);
    }
}
public class Firma {
    // Tablica na maksymalnie 100 pracowników
    private Pracownik[] listaPracownikow = new Pracownik[100];

    private int liczbaZatrudnionych = 0;

    // Metoda do dodawania nowego pracownika
    public void dodajPracownika(Pracownik nowyPracownik) {
        if (liczbaZatrudnionych < 100) {
            listaPracownikow[liczbaZatrudnionych] = nowyPracownik;
            liczbaZatrudnionych++;
            System.out.println("Zatrudniono nową osobę!");
        } else {
            System.out.println("Brak miejsc w firmie! Limit 100 pracowników
osiągnięty.");
        }
    }

    // Metoda wypisująca spisy
    public void wypiszSpis() {
        System.out.println("\n=== SPIS PRACOWNIKÓW FIRMY ===");
        if (liczbaZatrudnionych == 0) {
            System.out.println("Firma jeszcze nikogo nie zatrudnia.");
        } else {
            for (int i = 0; i < liczbaZatrudnionych; i++) {

                listaPracownikow[i].wypiszDanePracownika();
            }
        }
    }
}
```

```
=== SPIS PRACOWNIKÓW FIRMY ===
Firma jeszcze nikogo nie zatrudnia.
Zatrudniamy ludzi...
Zatrudniono nową osobę!
Zatrudniono nową osobę!
Zatrudniono nową osobę!

=== SPIS PRACOWNIKÓW FIRMY ===
Pracownik: Jan Kowalski | Stanowisko: Kierownik
Pracownik: Anna Nowak | Stanowisko: Główna Księgowa
Pracownik: Piotr Zieliński | Stanowisko: Młodszy Programista
Zatrudniono nową osobę!

=== SPIS PRACOWNIKÓW FIRMY ===
Pracownik: Jan Kowalski | Stanowisko: Kierownik
Pracownik: Anna Nowak | Stanowisko: Główna Księgowa
Pracownik: Piotr Zieliński | Stanowisko: Młodszy Programista
Pracownik: Krzysztof Krawczyk | Stanowisko: Stażysta
```

Tablica od razu tworzy 100 "pustych krzeseł" (czyli wartości null). Gdybyśmy puścili pętlę przez wszystkie 100 miejsc, żeby je wypisać, program wyrzuciłby błąd (NullPointerException), trafiając na puste miejsce. Zmienna liczbaZatrudnionych działa jak wskaźnik – mówi pętli, w którym momencie ma się zatrzymać.

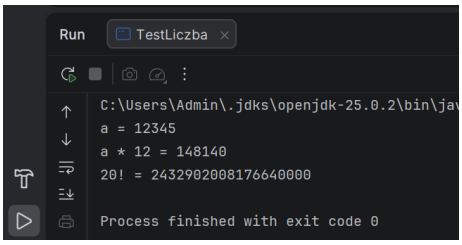
To kluczowa zasada programowania obiektowego. Klasa Firma nie powinna "wiedzieć", z jakich dokładnie pól składa się Pracownik (czy ma imię, nazwisko, czy wiek). Jedyne co Firma robi, to zarządza zbiorem (tablicą) i mówi: *"Hej pracowniku pod indeksem zero, przedstaw się!"*. Samą logikę budowania tekstu (kropki, spacje, etykiety) zamknęliśmy w metodzie wypiszDanePracownika() wewnątrz klasy Pracownik. Dzięki temu, jeśli w przyszłości dodamy pracownikowi pole "pensja", zmienimy kod tylko w jednym pliku!

**Zadanie 6.** Zdefiniuj klasę Liczba, która przechowuje w tablicy cyfry liczby dziesiętnej. Zdefiniuj operacje wypisywania liczby, nadawania jej wartości (w postaci parametru konstruktora będącego napisem) oraz mnożenia przez liczbę typu int. W przypadku gdy w czasie mnożenia okaże się, że tablica jest za mała, procedura mnożąca powinna kopiować jej zawartość do większej. Zdefiniuj wreszcie metodę silnia, która policzy silnię zadanej jako parametr liczby typu int.

```
public class Liczba {
    private int[] cyfry;   // cyfry w systemie dziesiętnym, cyfry[0] =
jedności
    private int rozmiar;   // faktyczna liczba używanych cyfr

    // Konstruktor z napisu, np. "12345"
    public Liczba(String s) {
        // usuwamy spacje
        s = s.trim();
        // pomijamy ewentualne wiodące zera
        int i = 0;
        while (i < s.length() - 1 && s.charAt(i) == '0') {
            i++;
        }
        s = s.substring(i);

        // tablica wielkości długości napisu
        cyfry = new int[s.length()];
        rozmiar = s.length();

        // zapisujemy cyfry w odwrotnej kolejności
        int idx = 0;
        for (int j = s.length() - 1; j >= 0; j--) {
            char c = s.charAt(j);
            if (c < '0' || c > '9') {
                throw new IllegalArgumentException("Nieprawidłowy znak w
liczbie: " + c);
            }
            cyfry[idx++] = c - '0';
        }
    }

    // Prywatna metoda zapewniająca miejsce w tablicy
    private void ensureCapacity(int minCapacity) {
        if (cyfry.length >= minCapacity) return;
        int newCap = Math.max(cyfry.length * 2, minCapacity);
        int[] nowa = new int[newCap];
        System.arraycopy(cyfry, 0, nowa, 0, rozmiar);
        cyfry = nowa;
    }

    // Mnożenie przez liczbę typu int (>= 0)
    public void pomnozPrzez(int m) {
        if (m < 0) {
            throw new IllegalArgumentException("Mnożenie przez liczbę
ujemną nieobsługiwane");
        }
        if (m == 0) {
            // wynik = 0
            cyfry = new int[1];
            cyfry[0] = 0;
            rozmiar = 1;
```

```
            return;
        }

        int carry = 0;
        for (int i = 0; i < rozmiar; i++) {
            long prod = (long)cyfry[i] * m + carry;
            cyfry[i] = (int)(prod % 10);
            carry = (int)(prod / 10);
        }

        // dopisujemy ewentualne cyfry z przeniesienia
        while (carry > 0) {
            ensureCapacity(rozmiar + 1);
            cyfry[rozmiar] = carry % 10;
            carry /= 10;
            rozmiar++;
        }
    }

    // Wypisanie liczby jako String
    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        for (int i = rozmiar - 1; i >= 0; i--) {
            sb.append(cyfry[i]);
        }
        return sb.toString();
    }

    // Metoda statyczna: silnia n (n!)
    public static Liczba silnia(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("Silnia niezdefiniowana dla
n < 0");
        }
        Liczba wynik = new Liczba("1");
        for (int i = 2; i <= n; i++) {
            wynik.pomnozPrzez(i);
        }
        return wynik;
    }
}
```

```
public class TestLiczba {
    public static void main(String[] args) {
        Liczba a = new Liczba("12345");
        System.out.println("a = " + a);          // 12345
        a.pomnozPrzez(12);
        System.out.println("a * 12 = " + a);     // 148140

        Liczba f = Liczba.silnia(20);
        System.out.println("20! = " + f);
    }
}
```




W zadaniu zaimplementowano klasę Liczba, która przechowuje wartość w postaci tablicy cyfr oraz umożliwia nadawanie wartości z napisu i wypisywanie jej w formie tekstowej. Dodatkowo zrealizowano mnożenie przez liczbę typu int z dynamicznym powiększaniem tablicy oraz metodę obliczającą silnię, co pozwoliło przećwiczyć pracę z dużymi liczbami bez użycia wbudowanych klas takich jak BigInteger.